Unravel: Root-Cause Analysis of RISC-V Build Failures via Long-Range Evidence Extraction and Rule-Tree Reasoning

Zehua Wang
Institute of Software, Chinese Academy of Sciences
Beijing, China
wangzehua23@otcaix.iscas.ac.cn

Zhirou Ma
Institute of Software, Chinese Academy of Sciences
Beijing, China
mazhirou@otcaix.iscas.ac.cn

Jie Liu*
Institute of Software, Chinese Academy of Sciences
Beijing, China
ljie@otcaix.iscas.ac.cn

Liangyi Kang
Institute of Software, Chinese Academy of Sciences
Beijing, China
kangliangyi15@otcaix.iscas.ac.cn

Shuai Wang
Institute of Software, Chinese Academy of Sciences
Beijing, China
wangshuai@otcaix.iscas.ac.cn

Dan Ye†
Institute of Software, Chinese Academy of Sciences
Beijing, China
yedan@otcaix.iscas.ac.cn

Jiaxin Zhu
Institute of Software, Chinese Academy of Sciences
Beijing, China
zhujiaxin@otcaix.iscas.ac.cn

Wei Wang
Institute of Software, Chinese Academy of Sciences
Beijing, China
wangwei@otcaix.iscas.ac.cn

Hua Zhong
Institute of Software, Chinese Academy of Sciences
Beijing, China
zhonghua@otcaix.iscas.ac.cn

Abstract

Porting open-source software to RISC-V frequently triggers build failures whose root causes are difficult to diagnose automatically. Build logs often span tens of thousands of lines, yet failure-relevant clues account for fewer than 5% of the content, and the true root cause may appear hundreds of lines before the terminal error. This creates long-range causal dependencies that naive local windowing easily misses. Meanwhile, historical repair knowledge is scattered across heterogeneous commits and issue discussions. Superficially similar error messages may require entirely different fixes depending on build stage, dependency version, and toolchain context, making pure similarity-based retrieval unreliable. We propose **Unravel**, a two-step framework that addresses both challenges through (1) Long-Range Evidence Extraction from Build Logs and (2) Rule-Tree Reasoning for Traceable Root-Cause Diagnosis. In the first step, Unravel combines rule-based error localization, entity-guided cross-stage backtracking, and lightweight semantic summarization to preserve high-fidelity evidence chains under strict token budgets. In the second step, Unravel distills structured repair rules from historical commits, organizes them in a rule-tree index by error type and build stage, and applies conditional routing with ReAct-style reasoning to generate traceable, evidence-grounded diagnoses. On a shared dataset of 127 real RISC-V build failures from a production OBS environment, evaluated with separate protocols for extraction and diagnosis, the first step improves recall and F1 over LogSage by 15.5% and 3.7%, respectively. The full Unravel pipeline reaches a composite diagnostic score of 84.0%; with DeepSeekV3, it improves overall score by 8.7% and reduces processing time by about 61% compared with MCTS.

*Corresponding author
†Corresponding author

CCS Concepts

• **Software and its engineering → Software testing and debugging**; *Software maintenance tools.*

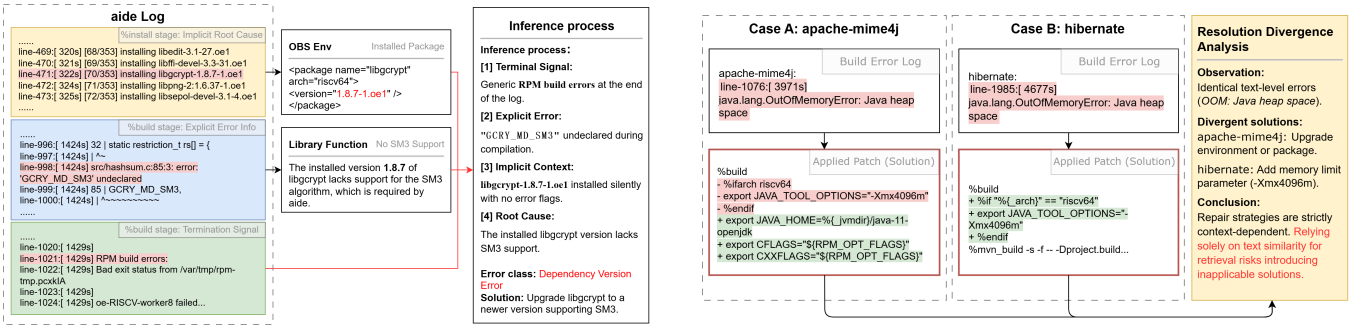
Keywords

RISC-V, build failure, root cause analysis, critical error extraction, large language model

1 Introduction

RISC-V has gained significant momentum in recent years, driven by its open instruction-set architecture, modularity, and extensibility [3], positioning it as a compelling alternative for next-generation general-purpose and domain-specific computing. As software migration advances, two practical realities have become increasingly clear. First, the number of failed packages is large in real migration workflows: large-scale CI studies report 34,182 failed builds across 767 projects (349 OSS projects and 418 projects in a financial organization) [23]. Second, migration is not a one-shot process. Even after a package is repaired once, upstream version changes, dependency updates, and toolchain evolution can re-trigger failures in later cycles [14]. In engineering practice, handling such recurring failures still relies heavily on manual root-cause analysis from build logs before developers can apply reliable fixes.

RISC-V build logs also expose two core technical difficulties. **Problem 1: long-range causal dependency in ultra-long logs.** Figure 1a shows a representative failure from `aide`: the build terminates with a generic `RPM build errors` signal near the tail (line 1021), and an explicit compile-time symptom appears at line 998 (`` `GCRY_MD_SM3' undeclared ``). However, the actual trigger is much earlier (line 471), where the dependency installation step introduces `libgcrypt-1.8.7-1.oe1`, whose feature set does not provide SM3 support. This head–middle–tail separation means that



(a) Long-range causal dependencies in build logs **(b) Similar errors but different fixes**

Figure 1: Motivating examples. (a) A build log with long-range causal dependency: the root cause at line 471 is far from the error at line 998. (b) Similarity misleading: two packages share the same error message but require different fixes.

root-cause evidence is distributed across distant regions and different build phases. If diagnosis keeps only a local window around the explicit error line, the decisive dependency-version clue is likely to be removed.

Problem 2: similarity misleading across failure cases. Figure 1b presents two logs (`apache-mime4j` and `hibernate`) that contain highly similar surface-level error messages, but their actionable causes differ. One case is rooted in package/dependency adaptation, while the other is tied to build-script and environment constraints; consequently, applying the same fix template to both cases leads to incorrect remediation in at least one of them. This example shows that successful repair depends not only on error text matching, but also on applicability conditions such as build stage, dependency state, architecture constraints, and toolchain configuration.

These issues make manual diagnosis costly and slow. Large language models (LLMs) have recently opened a new opportunity for automating build-failure diagnosis, but directly applying LLMs still faces two major challenges: **(C1)** build logs are often much longer than practical context budgets, and naive truncation breaks evidence chains; **(C2)** LLMs lack stable RISC-V build-domain knowledge, and vector-similarity-based RAG can introduce misleading historical cases when applicability conditions are not explicitly modeled.

To address these challenges, we propose **Unravel**, a unified framework with two tightly coupled methods: **Long-Range Evidence Extraction from Build Logs** and **Rule-Tree Reasoning for Traceable Root-Cause Diagnosis**. The first method preserves high-value evidence under strict token budgets; the second method organizes historical repair knowledge into condition-aware rules and performs traceable routing-based diagnosis.

To address the above challenges, this paper makes the following contributions:

(1) We propose **Long-Range Evidence Extraction from Build Logs**, which improves recall and F1 over LogSage by 15.5% and 3.7%, respectively.
(2) We propose **Rule-Tree Reasoning for Traceable Root-Cause Diagnosis**, which reaches an overall diagnostic score of 84.0% and improves score by 8.7% over MCTS while reducing processing time by about 61%.
(3) We integrate these two methods into the **Unravel** framework and deploy it in the **RuiQian toolset plugin** for practical RISC-V migration support.

2 Related Work

2.1 Log Parsing and Build-Log Evidence Extraction

Log parsing seeks to transform unstructured log data into structured representations comprising event templates and dynamic parameters. Early approaches relied on rules and regular expressions. Representative methods include SLCT [22], which mines log patterns via clustering, Drain [8], which constructs a fixed-depth parsing tree for efficient online parsing, and Logram [4], which employs n-gram dictionaries to accelerate template recognition. With the advent of deep learning, LSTM-based methods such as DeepLog [5] learn temporal patterns of normal execution paths for anomaly detection, while Transformer-based and self-supervised approaches yield stronger semantic representations [2, 7]. Semi-supervised methods such as PLELog [28] reduce annotation overhead, whereas LogGPT [17] and LogLLM [6] directly leverage pre-trained language models for anomaly detection. Nevertheless, these methods often struggle to generalize across heterogeneous systems and evolving software versions [9].

More recently, large language models (LLMs) have introduced a new paradigm for log parsing [1]. LogPrompt [13], a few-shot log-parsing method, reduces cost through judicious example selection; DivLog [26] enhances cross-domain parsing via in-context learning; LLMParser [15] treats LLMs as generative parsers and provides systematic empirical analysis. LogShrink [11] focuses on long-log streamlining through commonality and variability patterns. LogSage [27], a log analysis framework for CI/CD scenarios, proposes token-efficient preprocessing and structured diagnosis. However, in build-failure scenarios involving software packages—where long-range causal dependencies and cross-stage reasoning are paramount—how to reliably preserve traceable evidence chains within stringent token budgets remains an open challenge necessitating task-specific extraction strategies.

From the build-failure perspective, prior empirical studies report frequent and diverse failures in CI and package-migration workflows [18], while recent engineering studies explore automatic build-error categorization to reduce manual triage effort [10]. These efforts provide useful operational insights, but they still leave open the key challenge of extracting high-fidelity, long-range causal evidence from ultra-long build logs under strict context budgets.

2.2 Log-based Root Cause Analysis

Root cause analysis (RCA) seeks to identify the underlying causes of failures from system outputs. Traditional approaches employ rule-based correlation to match events [16], or combine log analysis with source-code examination to infer failing execution paths [29]. Deep learning methods formulate RCA as a log-line relevance ranking problem, extracting indicative descriptions and parameters from logs for structured diagnosis [25]. Knowledge-based approaches organize failure patterns into structured representations: LogKG [21] constructs knowledge graphs from log events for structured diagnosis, and Squeeze [12] performs multi-dimensional root cause localization.

LLM-based RCA emphasizes generating human-readable diagnostic conclusions directly from failure logs. Roy et al. [19] demonstrate that agent-based designs can mitigate the sensitivity of single-turn question answering to context budgets and incomplete information. RCAgent [24] further validates the effectiveness of tool-augmented LLMs for cloud-based RCA, and PACE-LM [30] explores calibrated confidence estimation with GPT-4 for incident analysis. Shuai et al. [20] propose a two-stage framework in which RVLAD performs key-evidence extraction and MCTS-RCA combines Monte Carlo tree search with RCA to guide LLM reasoning. However, approaches that rely solely on "similar-log retrieval + free-form generation" risk introducing misleading evidence and amplifying hallucinations; applicability conditions, evidence alignment, and verification mechanisms must be systematically incorporated. Our method explicitly distills verifiable rules from historical repair experience and organizes them into a rule tree with conditional routing, thereby mitigating the risk of misleading retrieval at its source.

Despite this progress, existing build-failure RCA pipelines still largely rely on a "short window + local features" paradigm: they either depend on handcrafted heuristics that are brittle across heterogeneous logs and evolving software versions, or directly truncate long logs before reasoning, which breaks cross-stage causal chains. In addition, many systems emphasize error-type prediction or loosely constrained recommendations, without explicitly modeling applicability conditions in historical repair knowledge. These gaps motivate our framework design: preserving long-range evidence in Step 1 and enforcing condition-aware, traceable rule-tree reasoning in Step 2.

3 Method

We adopt a two-step pipeline for analyzing RISC-V build failures. In the first step, Long-Range Evidence Extraction from Build Logs performs evidence extraction on ultra-long logs, producing a structured summary centered on key error lines with the necessary causal context. In the second step, Rule-Tree Reasoning for Traceable Root-Cause Diagnosis consumes the structured extraction result, combines it with a rule-tree index constructed from historical repair experience, and generates traceable root cause analyses and repair recommendations via conditional routing and ReAct-style reasoning.

3.1 Extraction Stage: Long-Range Evidence Extraction from Build Logs

Evidence extraction for build-failure diagnosis is not merely about dropping redundant lines; the goal is to preserve the critical evidence required for root cause analysis under a constrained token budget. In Unravel, this step is designed for long-range causal dependencies in build logs. The key idea is to (1) quickly localize failure manifestations using explicit error anchors, (2) backtrack triggering conditions over long distances using key entities, and (3) perform fine-grained semantic summarization over candidate evidence. This process transforms the original log into a causally informative structured summary. This step follows two design principles: multi-granularity and long-range association. Multi-granularity progressively narrows focus from key error lines to related segments and then to entity-level clues; long-range association reconnects distant causal evidence through entity backtracking, which would otherwise be lost under tail-only windowing. As shown in Figure 2, this step includes three sub-steps: Step 1 applies lightweight rules and heuristic scoring to select high-recall candidate error blocks; Step 2 extracts key entities and backtracks them over the full log to recover causal chains; Step 3 performs fine-grained semantic summarization and structuring to produce a high-density representation under a fixed budget.

Given an original build log $L = \{l_1, l_2, \dots, l_N\}$, the goal of Step 1 is to produce a structured extraction result S that satisfies a token-budget constraint $|S| \leq B$ while maximizing the coverage of key evidence and causal clues in S. Here B is typically set to half of the downstream LLM's context window, leaving sufficient space for subsequent root cause reasoning and tool calls.

3.1.1 Fast Coarse-Grained Filtering. RISC-V package build logs on OBS are characteristically ultra-long, heterogeneous, and noisy. On one hand, logs contain voluminous routine compilation outputs, platform diagnostics, and toolchain metadata, while genuinely root-cause-relevant content constitutes only a minute fraction, yielding a very low signal-to-noise ratio. On the other hand, the build process spans multiple stages, and triggering conditions may first surface during early probing or dependency-checking stages yet manifest as explicit errors only later during compilation or linking. Applying entity backtracking or semantic summarization directly to the full log is not only computationally prohibitive and token-intensive but also susceptible to interference from redundant information. Therefore, Step 1 first performs fast coarse-grained filtering, which quickly removes verified useless outputs using lightweight rules and heuristics, locates explicit error anchors, and generates a small set of highly relevant candidate blocks centered around those anchors.

Concretely, we first apply a set of regular-expression rules R_{drop} to eliminate redundant outputs that recur across packages but bear no relevance to failures (e.g., compiler version echoes, download

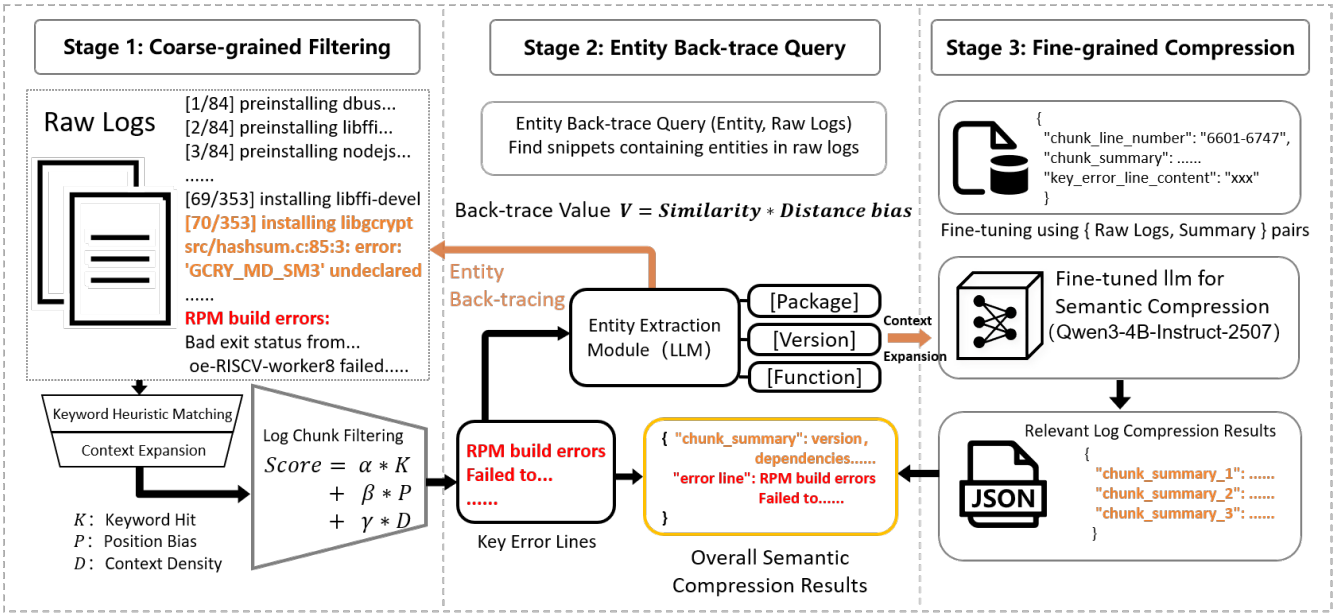


Figure 2: Overall workflow of Unravel Step 1: Long-Range Evidence Extraction from Build Logs

progress bars, fixed-format status messages), yielding a filtered log $L' = \{l_i \in L \mid g(l_i) = 1\}$. We then perform multi-level keyword matching to assemble a set of key error lines A, with matching tiers including: (1) strong error patterns such as `error:`, `fatal`, `undefined reference`, `No such file`, and `Bad exit status`; (2) build termination messages such as `RPM build errors` and `make: *** Error`; and (3) supplementary signals such as `undefined symbol` and `cannot find -lxxx`. Duplicate errors and repeated templates are merged to prevent a single error from consuming an excessive portion of the budget.

After obtaining the key error lines A, we expand a window around each error line to form candidate blocks $\mathcal{B}$, ensuring local semantic completeness. For each line l_i in a block, we compute a line-level importance score:

$$S(l_i) = \alpha \cdot I_{key}(l_i) + \beta \cdot f_{pos}(i) + \gamma \cdot D_{context}(i) \tag{1}$$

where I_{key} is an indicator function for keyword matches, $f_{pos}(i) = e^{-\lambda(N-i)/N}$ is a positional bias term (lines closer to the log tail receive higher weights), and $D_{context}$ is a local keyword-density term. Block-level scores are obtained by aggregating line scores (e.g., max + mean), ensuring that blocks containing strong error anchors are ranked higher while avoiding an unfair advantage for longer blocks. The system ranks blocks by their scores and selects the Top-n blocks under the token budget. In our setting, the total token length of this step is constrained to be no more than one quarter of the downstream LLM's context window, leaving ample room for subsequent entity backtracking and semantic summarization.

3.1.2 Entity-Based Context Backtracking. After coarse-grained filtering, the system has obtained a set of highly relevant candidate blocks anchored by explicit error lines that preserve the local context of failure manifestations. However, in RISC-V build-failure scenarios, relying solely on windowed snippets surrounding errors is frequently insufficient for reliable root cause analysis due to pronounced long-distance effects: the true triggering clues may appear earlier in `configure` checks, dependency installations, or configuration information, while final errors break out during `%build`; the same root cause may manifest multiple times in different forms, for example, an early warning about missing headers/libraries, later compilation errors about undefined types/symbols, and final link-time `undefined reference` messages. This "cause first, symptoms later" long-range causal structure leads to broken evidence chains if we only preserve local windows around explicit errors.

To complete long-range causal chains, this step extracts backtrackable query elements from explicit error segments and organizes them into a structured entity set E. Here, entities do not refer to arbitrary noun phrases in natural language but rather to symbolic clues that recurrently appear in build logs and point to key nodes in failure chains. These include function and symbol names (e.g., `__cpuid`, `riscv_v_asm_flag`), library names (e.g., `libatomic`, `openssl-devel`), missing headers, target files and paths, build targets, macros, and critical compiler or linker flags. The quality of E directly governs backtracking effectiveness: an insufficiently populated entity set leads to inadequate recall, while an overly noisy set introduces irrelevant segments.

Using the entity set E as queries, we search the original log L for related segments. For each line l_j, we define a backtracking value:

$$V(l_j) = \max_{e \in E} \left(\mathrm{Sim}(l_j, e) \cdot \frac{1}{\log(1 + \lambda \Delta t(l_j, e))} \right) \tag{2}$$

where Sim denotes fuzzy matching similarity (supporting substring matches, case-insensitive matches, and small edit distances), and the distance term biases toward closer lines when similarities are comparable. Lines whose values exceed a threshold are selected

and aggregated into backtracking segments, thereby filling in causal evidence scattered across build stages. For example, when an explicit error reports `undefined reference to `foo_v_probe'`, entity backtracking may locate, in earlier `configure` stages, lines like `checking for foo_v_probe... no` and the corresponding compilation failures in `config.log`, thus bridging configuration-probing failures and link-time errors.

3.1.3 Fine-Grained Semantic Summarization. After coarse-grained filtering and entity backtracking, the system has converged on two categories of valuable information: key error lines with their local context, and related segments recovered by backtracking entities across the entire log. However, these backtracking results still exist as raw log segments that may contain redundancy, exhibit stylistic inconsistency, and exceed acceptable length, risking overflow of the downstream model's context capacity. Naively concatenating them as LLM input would incur substantial token overhead and introduce noise and duplication that degrade the stability of subsequent reasoning.

Therefore, this step performs fine-grained semantic summarization. We use a lightweight, domain-tuned model to summarize and distill backtracking segments and then fuse them with explicit error information preserved in the coarse-grained step, producing a structured and budget-controlled semantic summary of the log. Here, the summarization is task-oriented rather than generic: it focuses on information that supports RCA. On the one hand, it preserves context lines/segments helpful for localization; on the other, it attaches stage labels (e.g., `%prep`, `%build`) and distills key facts directly relevant to root causes, such as certain dependencies being disabled in `configure` due to missing packages, link-time failures like `cannot find -lxxx`, or architecture-specific symbols not being available under RISC-V.

We employ Qwen3-4B-Instruct as the base model, which exhibits strong instruction-following and text-generation capabilities at a moderate parameter scale suitable for cost-effective deployment. We apply supervised fine-tuning (SFT) with LoRA-based parameter-efficient tuning on 2,925 manually curated log-block extraction examples to guide the model toward preserving critical information, reducing uninformative content, and producing stable output formats. In Unravel Step 1, except for this fine-tuned summarization model, all other LLM invocations use DeepSeekV3. In addition, the 2,925-example fine-tuning dataset is fully independent from the evaluation dataset reported in Section 4. Experiments demonstrate that key-error coverage increases from 74.3% to 75.7%, confirming that domain-specific tuning improves extraction quality while maintaining concise and structured outputs.

Finally, we fuse the fine-grained summaries with explicit error information from the coarse-grained stage to obtain the structured extraction result S, which is human-readable, reproducible, and well-suited for subsequent rule-based root cause reasoning.

3.2 Reasoning Stage: Rule-Tree Reasoning for Traceable Root-Cause Diagnosis

To mitigate the similarity misleading inherent in naive similarity-based retrieval and to ensure that repair recommendations are verifiable with explicit evidence, we design Step 2 as rule-tree-based diagnosis. The fundamental idea is to extract and explicitly structure the relationships among "symptoms—applicability conditions—repair actions—evidence" from historical repair records, transforming scattered experience into reusable repair rules, and to construct a maintainable rule-tree index along axes of problem category, build stage, and verifiable conditions. During online inference, rather than retrieving by similarity directly, we use the Step-1 summary as input for conditional routing over the rule tree, activating rules only when their applicability conditions are satisfied, and subsequently generating root cause explanations and fixes under explicit evidence constraints. This paradigm yields interpretable, traceable, and actionable diagnostic outputs.

As shown in Figure 3, the overall workflow contains an offline construction and an online inference part. Offline, we process historical repair commits to extract and structure repair experience from commit messages, diffs/patches, and packaging-related spec changes, and store them as reusable rules. Then, we construct a rule-tree skeleton organized by error category and build stage, and attach each rule to leaf nodes whose conditions are satisfied, with conflict detection and incremental updates. Online, we take the Stage-1 summary S (optionally augmented with spec/patch and environment summaries) as input, first determine error type and build stage to locate the entry in the rule tree, and then traverse conditional branches to retrieve candidate leaf rules. Under evidence constraints, we finally generate a diagnostic report containing "root cause—fix—applicability conditions—evidence references".

3.2.1 Structuring Historical Repair Information. In practice, historical repair experience is primarily recorded in version-control commits, encompassing commit messages, code diffs/patches, and packaging-related spec changes. Although these artifacts contain rich repair knowledge, they are inherently unstructured, stylistically heterogeneous, and varied in evidence granularity: the same category of problem may be described in disparate terms (e.g., "fix build", "riscv workaround", "linker error"), while the actual executable repair actions are often embedded within diff fragments. Directly using raw commits as retrieval targets in similarity-based methods readily reintroduces similarity misleading and fails to explicitly capture the relationship between applicability conditions and repair actions. Therefore, Step 2 first abstracts historical commits into structured units—repair rules—thus elevating experience from free text to computable, routable, and maintainable knowledge items.

Rule construction takes a single repair commit as the basic object, with inputs including: (1) commit message describing intent and background; (2) diffs/patches containing concrete code or script changes; (3) spec changes related to packaging (e.g., dependency declarations, macro and build-parameter modifications, patch references); and (4) associated issue/PR discussions providing information about reproduction, applicability conditions, and boundary cases. The output is a structured JSON record whose fields emphasize two aspects. First, applicability conditions are explicitly encoded (e.g., `error_type`, `build_stage`, `trigger_condition`, `risc_v_specific`, `can_generalize`) to support later tree construction and conditional routing. Second, evidence carriers (e.g., `files_touched`, `commit_id`, `notes`) are preserved to ensure traceability and human verification.

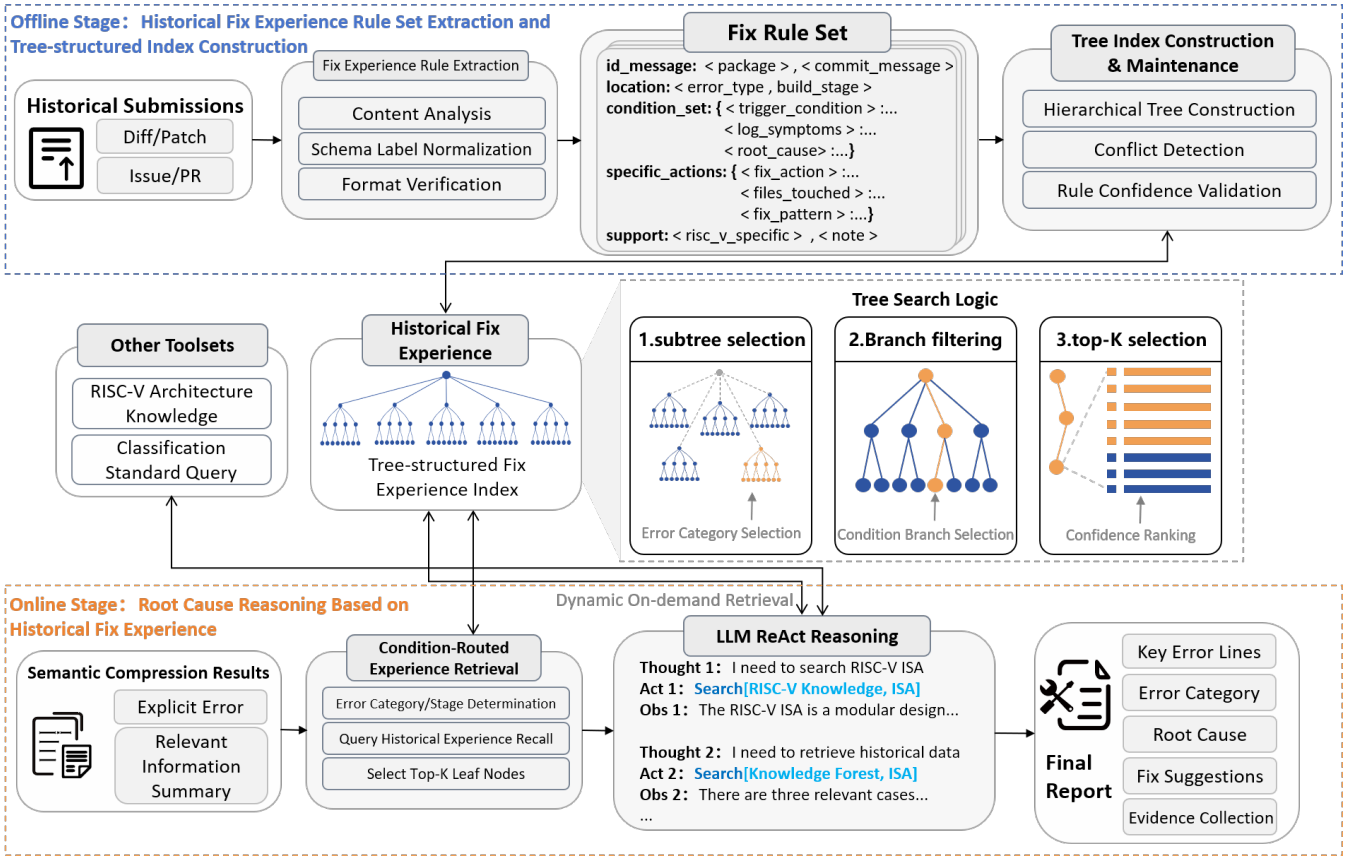


Figure 3: Unravel Step 2: Rule-Tree Reasoning for Traceable Root-Cause Diagnosis

The construction pipeline adopts a three-stage process: "traditional parsing and preprocessing—LLM-based structuring—validation and normalization". First, we use traditional techniques to pre-parse metadata and changes, including segmenting commit messages, diffs/patches, spec changes, and associated issue/PR content, and extracting salient elements such as touched files, key change hunks, dependency declarations, and macro/parameter modifications. This reduces input noise and provides structural hints for LLMs. Second, with unified prompt templates and fixed output schemas, we invoke an LLM to convert parsed results into structured repair rules, covering fields such as `error_type`, `build_stage`, `trigger_condition`, `root_cause`, and `fix_action`, and mapping key fields such as triggers and fix patterns to predefined label sets whenever possible. Finally, an independent validation module automatically checks LLM outputs: it verifies schema conformance (field completeness, type consistency, non-empty required fields) and performs normalization consistency checks for `fix_action` and `trigger_condition`. When validation fails, the system triggers a rewrite strategy, asking the model to regenerate specific fields using feedback from validation, ensuring that only rules with stable formats, routable conditions, and traceable evidence enter the knowledge base.

3.2.2 Rule-Tree Index Construction and Maintenance. After obtaining structured rules, we must organize the dispersed repair experience into a routable and maintainable knowledge structure. We adopt a tree index for knowledge organization for three reasons. First, the manual diagnostic workflow for build failures inherently follows a decision path of first determining error category and build stage, then narrowing down by applicability conditions. A tree index explicitly encodes this process as conditional branches, rendering rule-matching paths naturally interpretable and auditable. Second, the effectiveness of repair experience is strongly conditioned on prerequisites (e.g., software versions, target architecture, dependency states, build systems, and build stages). Tree routing mandates that each step satisfy verifiable conditions, filtering out inapplicable rules at the retrieval entry point and thereby mitigating the risks of erroneous retrieval due to superficial similarity. Third, supplying all rules and their conditions to an LLM for ad-hoc conditional retrieval would entail prohibitive token costs, and the model would be susceptible to attention dispersion and overlooked conditions amid the resulting redundancy. The tree index pre-materializes the routing structure, performing structural pruning prior to LLM inference and injecting only the small subset of rules that satisfy the current conditions, thus enabling compact-context, strongly constrained, and auditable reasoning.

As illustrated in Figure 4, we start from a root node and split the first layer into five domains in accordance with our error taxonomy (architecture compatibility, version compatibility, dependency configuration, compilation configuration, and other errors), so that online inference can first perform coarse routing at the problem-domain level. The second layer is organized by build stage (e.g., `%prep`, `%build`, `%install`, `%check`), constraining historical experience to consistent stage contexts to avoid misinterpretation of the same error text under different stages. Deeper nodes branch according to verifiable routing conditions derived from `trigger_condition` and `log_symptoms` in rules, such as "whether dependency xxx is detected missing", "whether the failure only occurs on RISC-V", etc. Leaf nodes store sets of concrete repair rules with their `fix_action`, applicability conditions, and annotations. Once a path is hit during online inference, the system obtains a joint package of "conditions—actions—evidence".

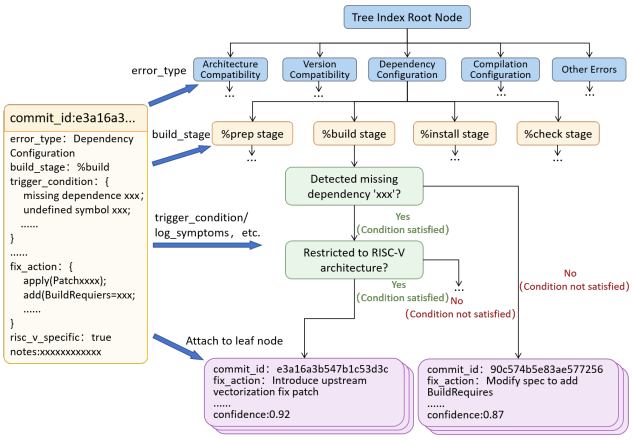


Figure 4: Illustration of the rule-tree index structure

To support continuous evolution, we design incremental updates and conflict-resolution mechanisms. Given a new rule, we first normalize it using the same offline procedure and then determine its routing position. If the condition set C_{new} of the new rule and C_{old} of an existing rule have inclusion relationships, we handle them as follows: when $C_{new} \supset C_{old}$, the new rule has stricter conditions, so we add a deeper branch in the tree and give the fine-grained rule higher priority; when $C_{new} \subset C_{old}$, the new rule is more general, and we adjust the tree to generalize conditions and reduce duplicate branches. This avoids naive stacking of rules that would lead to path explosion.

We further introduce rule confidence for quality control:

$$\mathrm{Conf}(r) = \alpha \cdot S_{evi}(r) + (1-\alpha) \cdot S_{time}(r) \tag{3}$$

where $S_{evi}(r) = n_r/(n_r + k)$ is evidence support (with n_r being the number of supporting records and k a smoothing constant, default k=3), and $S_{time}(r) = 1/(1 + (t_{now} - t_r)/T)$ measures recency (with T=90 days, meaning the recency drops to 0.5 after 90 days). Rules whose confidence falls below a threshold are used only as low-priority candidates and placed in a queue for manual review; only those above the threshold become high-priority leaf evidence for routing. This mechanism jointly constrains knowledge quality and freshness during incremental evolution.

3.2.3 Online Root Cause Reasoning. In the online phase, given a build-failure instance, the objective is to perform conditional routing and historical-experience retrieval along the rule tree, and then, under explicit evidence constraints, generate interpretable, actionable, and verifiable root cause analyses and repair recommendations. We adopt a ReAct-style evidence-driven reasoning framework, employing the Stage-1 summary S as the entry point and encapsulating external knowledge queries and rule-tree retrieval as invocable tools. The model then iterates between "acting" and "observing" to progressively accumulate evidence and converge on applicable rules.

Error type and stage determination. The first step is to determine the error category and build stage of the current failure. The Stage-1 summary S already encodes explicit error lines, stage-related clues, key entities, and supporting evidence snippets. If S explicitly contains stage information, we use it directly; otherwise, we infer the stage from characteristic patterns surrounding the error anchors. For error categorization, we map error anchors and entities in S to one of our five predefined categories via a standardized decision procedure. When S reveals architecture-specific entities or patterns, we optionally invoke RISC-V knowledge tools for supplementary verification to prevent misclassification based solely on surface-level error messages.

Retrieving relevant historical repair experience. After determining error type and stage, we query the rule tree. We first locate the appropriate problem-domain node according to `error_type` and then descend into the corresponding stage subtree according to `build_stage`, confining candidates to a semantically coherent region. Within the stage subtree, we use triggers and key entities extracted from S to evaluate branch conditions. We traverse top-down: when evidence unambiguously satisfies a branch condition, we follow it; when evidence is insufficient for a definitive judgment, we retain the branch as a candidate to avoid premature pruning; when evidence explicitly contradicts a branch, we skip it. If multiple leaf nodes remain, we rank their rules by matching degree (coverage of triggers and entities) and rule confidence, retaining the Top-K rules as candidate evidence. If no leaves are matched, we back off to higher-level nodes and fall back to more general rules.

ReAct-style reasoning. Upon retrieving candidate rules, we proceed to diagnostic output generation. Three categories of information are assembled in the LLM context: (1) the Stage-1 summary S, providing error anchors, stage clues, and entities; (2) retrieved historical rules (Top-K candidates); and (3) routing path metadata (problem domain, stage, and branch sequence). Following the ReAct paradigm, the model first formulates preliminary hypotheses based on S and the candidate rules. If it identifies missing critical information or inconsistencies among candidates, it issues "actions" to invoke tools (e.g., RISC-V knowledge queries or validation of hypothetical fixes), incorporates tool outputs as "observations," and then refines its hypotheses accordingly. The model is required to produce a structured diagnostic report comprising: (1) key error and stage localization; (2) error-type classification with rationale; (3) a verifiable root cause explanation; (4) concrete, executable repair recommendations; and (5) supporting evidence with references to historical repairs. When multiple candidate rules remain plausible, the system outputs a primary

recommendation alongside alternatives, each annotated with its `trigger_condition` delineating applicability boundaries, thereby avoiding unverifiable compromises when evidence conflicts.

4 Evaluation

We evaluate the proposed framework from four complementary perspectives:

- **RQ1 (Extraction Effectiveness):** How well does Step 1 preserve diagnosis-critical evidence compared with representative extraction baselines?
- **RQ2 (Extraction Ablation):** How much does each Step-1 module contribute to final extraction quality?
- **RQ3 (Diagnosis Effectiveness):** How does Unravel perform against mainstream retrieval- and search-based RCA baselines?
- **RQ4 (Efficiency):** Can Unravel maintain practical latency while improving diagnostic quality?

4.1 Experimental Setup

4.1.1 Datasets. Our experiments are conducted on real build-failure logs collected from a production OBS environment for RISC-V package migration. We use a shared dataset of 127 build-failure logs (149,219 lines in total) for both steps of Unravel, but evaluate them with separate protocols: Step 1 is evaluated as an evidence-extraction task, while the full pipeline is evaluated as a root-cause-analysis task. For Step-1 evaluation, domain experts annotate 4,582 repair-relevant lines as ground truth. For full-pipeline evaluation, each sample in this same 127-case dataset is evaluated as one failure case. The five RCA categories are architecture compatibility, version compatibility, dependency configuration, compilation configuration, and other errors. The Step-1 fine-tuning dataset (2,925 examples) is collected and split independently, with no overlap with this 127-case evaluation dataset.

Figure 5 shows that version-compatibility errors are the most frequent (40/127, 31.5%), followed by dependency-configuration errors (35/127, 27.6%), compilation-configuration errors (24/127, 18.9%), other errors (17/127, 13.4%), and architecture-compatibility errors (11/127, 8.7%). In terms of build stages, failures are concentrated in `%build` (59/127, 46.5%), followed by `%prep` (38/127, 29.9%), `%check` (23/127, 18.1%), and `%install` (7/127, 5.5%).

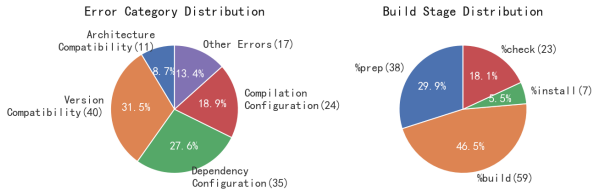


Figure 5: Distribution of error categories and build stages in the RISC-V build-failure dataset

4.1.2 Baselines. For evidence extraction, Unravel Step 1 is compared with three representative baselines: (1) LogSage [27], an LLM-based long-log analysis framework; (2) TemplateRAG [20], a template parsing plus retrieval-augmented method; and (3) LogPrompt [13], a prompt-engineering strategy for LLM-based log analysis.

For root cause analysis, we include four baseline groups: (1) naive RAG with semantic retrieval plus generation; (2) LogSage [27] with multi-path retrieval enhancement; (3) MCTS [20] as a search-based reasoning framework; and (4) standalone flagship reasoning models (Seed 1.8[1], Kimi K2[2], DeepSeek-V3.2), which emphasize stronger native reasoning capability.

4.1.3 Implementation Details. For Unravel and comparable baselines, we use DeepSeekV3 and GPT-OSS as backbones. Standalone reasoning models are evaluated end-to-end using Step-1 structured extraction outputs as inputs. The Step-1 extractor is fine-tuned from Qwen3-4B-Instruct with LoRA-based parameter-efficient tuning (Section 3.1). The rule-tree knowledge base is built offline from historical OBS repair commits using the pipeline in Section 3.2.

4.1.4 Metrics. For critical-error extraction, we report line-level Precision, Recall, and F1 to measure key-evidence preservation quality. For root cause analysis, we use the following composite score:

$$\text{score}(i) = \frac{s_i + c_i + r_i + f_i}{4} \tag{4}$$

where s_i denotes key-error extraction quality, c_i error-type classification quality, r_i root-cause explanation quality, and f_i repair recommendation quality. Each sub-score ranges from 0 to 1 and is independently rated by three domain experts using a five-level rubric: 1.0 (fully correct), 0.75 (mostly correct with minor omissions), 0.5 (partially correct), 0.25 (mentioned but largely incorrect), and 0 (missing or incorrect). The final sub-score is determined by majority voting. Inter-annotator agreement reaches Fleiss' $\kappa = 0.72$, indicating substantial consistency.

4.2 RQ1: Effectiveness of Unravel Step 1 for Evidence Extraction

Table 1: Accuracy comparison of critical-error extraction methods

Method	Prec	Rec	F1
LogPrompt	0.262	0.305	0.282
TemplateRAG	0.047	**0.764**	0.089
LogSage	**0.314**	0.581	0.408
Unravel Step 1 (ours)	0.308	0.671	**0.423**

Table 1 shows that Unravel Step 1 achieves the best overall balance among the compared methods: it obtains the highest F1 score (0.423) while maintaining high recall (0.671). Compared with LogSage, Step 1 improves recall by approximately 15.5% and F1 by about 3.7%, indicating that it preserves more key repair evidence without introducing excessive noise. TemplateRAG reaches the highest recall (0.764) but suffers from extremely low precision (0.047),

[1]Seed 1.8: a reasoning model developed by ByteDance.
[2]Kimi K2: a reasoning model developed by Moonshot AI.

suggesting heavy over-retention of irrelevant content. In contrast, Step 1 keeps precision and recall in a more practical range for downstream diagnosis. In terms of token efficiency, Step 1's average output lengths for three log groups are 3442/4234/5502 tokens, far smaller than TemplateRAG (38746/49883/55406), leaving substantially more context budget for the reasoning step. Runtime results also show that Step 1 provides about 2.3–3.6× acceleration over LogPrompt and TemplateRAG on ultra-long logs.

4.3 RQ2: Contribution of Each Step-1 Component

We perform an ablation study to assess the contribution of each Step-1 component.

Table 2: Ablation results of Unravel Step 1

Configuration	Prec	Rec	F1
Step 1 (full)	0.308	0.671	0.423
w/o Anchor Filter	0.274	0.352	0.308
w/o Entity Backtracking	0.292	0.584	0.389
w/o Semantic Summarizer	0.216	0.671	0.327

The ablation results in Table 2 confirm that each Step-1 component contributes to final performance. Removing the anchor filter causes the largest degradation, with recall dropping from 0.671 to 0.352 and F1 dropping from 0.423 to 0.308, which highlights the importance of early anchor-based localization. Removing entity backtracking leads to consistent declines in recall (0.584) and F1 (0.389), showing that cross-stage clue recovery is essential for maintaining evidence continuity. When the semantic summarizer is removed, recall remains unchanged (0.671), while precision decreases markedly (0.216 vs. 0.308), leading to a lower F1 (0.327 vs. 0.423). This indicates that removing the summarizer mainly retains more content and therefore does not hurt recall, but it introduces more noise and reduces output purity.

4.4 RQ3: Comparison with Baselines on Root Cause Analysis

Table 3: Overall performance comparison on root cause analysis

Group	Method	s_i	c_i	r_i	f_i	Score
DeepSeekV3	RAG	0.943	0.793	0.799	0.635	0.793
	LogSage	0.912	0.764	0.718	0.546	0.735
	MCTS	0.936	0.809	0.749	0.596	0.773
	Unravel (ours)	**0.969**	**0.828**	**0.814**	**0.751**	**0.840**
GPT-OSS	RAG	**0.915**	0.773	**0.851**	0.750	0.822
	LogSage	0.839	0.726	0.762	0.638	0.741
	MCTS	0.913	0.788	0.819	0.635	0.789
	Unravel (ours)	0.908	**0.789**	0.829	**0.794**	**0.830**
Reasoning Models	Seed 1.8	0.933	0.765	0.849	0.654	0.800
	Kimi K2	**0.959**	**0.804**	0.854	0.674	0.822
	DSV3.2	0.935	0.777	**0.884**	**0.693**	0.822

Table 3 shows that Unravel delivers the strongest overall RCA performance among methods using the same backbone. Under DeepSeekV3, Unravel achieves the best total score (0.840), surpassing MCTS (0.773) by about 8.7% and RAG (0.793) by about 5.9%. The largest gains appear in repair suggestion (f_i = 0.751), where Unravel clearly exceeds all baselines, indicating that rule-conditioned retrieval improves the executability of generated fixes. Under GPT-OSS, Unravel also reaches the highest score (0.830), slightly above RAG (0.822), while maintaining the best repair recommendation quality (f_i = 0.794). These results suggest that structured historical knowledge remains beneficial even when backbone model capability is strong.

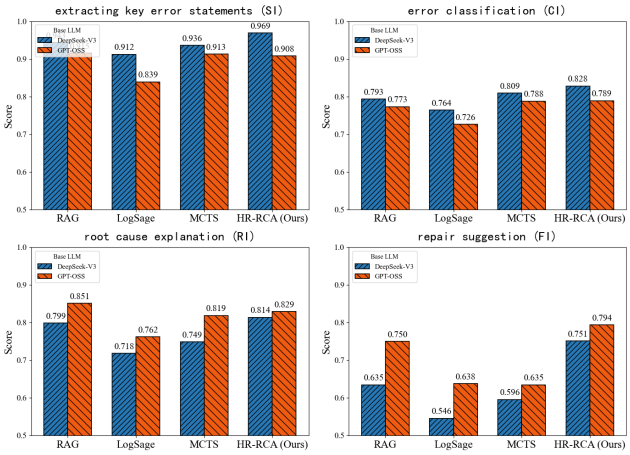


Figure 6: Per-metric score comparison across methods

Figure 6 further illustrates that Unravel's advantage is most pronounced in the downstream reasoning dimensions, especially repair recommendation and root cause explanation. Under DeepSeekV3, Unravel improves f_i from 0.596 (MCTS) to 0.751 and raises r_i to 0.814, indicating that evidence-constrained routing helps the model connect observed failures with historically validated fixes. Meanwhile, all methods obtain relatively high scores on key-error extraction, suggesting that Step 1 already provides stable error anchors for subsequent reasoning. The remaining performance gap is therefore mainly determined by how effectively each method organizes evidence and generates actionable repair paths.

Compared with standalone flagship reasoning models, the Unravel pipeline remains highly competitive. The best standalone scores are 0.822 (Kimi K2 and DSV3.2), while Unravel with GPT-OSS reaches 0.830. This result indicates that, for build-failure diagnosis, explicit rule constraints and historical evidence grounding can still provide measurable gains beyond relying only on larger general-purpose reasoning models.

4.5 RQ4: Efficiency Analysis

As shown in Figure 7, Unravel achieves a favorable effectiveness-efficiency trade-off. With DeepSeekV3, its average runtime is 101.80s, which is about 61.0% faster than MCTS (261.23s) and close to LogSage (86.93s). Although naive RAG is the fastest (32.64s), its diagnostic quality is notably lower. These results indicate that Unravel avoids the heavy cost of global search by narrowing candidates

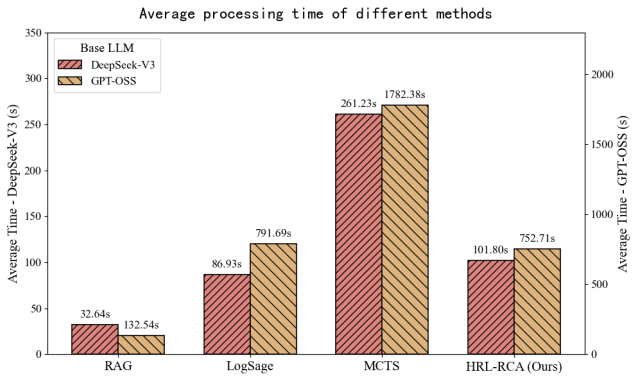


Figure 7: Average processing time of different methods

through rule routing, while retaining stronger diagnostic quality than purely retrieval-based alternatives.

5 Conclusion and Future Work

This paper addresses build failures in RISC-V package migration by proposing Unravel, a two-step root cause analysis framework that combines Long-Range Evidence Extraction from Build Logs with Rule-Tree Reasoning for Traceable Root-Cause Diagnosis. In Step 1, Unravel applies rule-based error localization, entity-based cross-stage backtracking, and lightweight semantic summarization to preserve high-fidelity evidence chains under strict token budgets, substantially reducing the input size of ultra-long logs. In Step 2, Unravel structures historical repair experience into verifiable rules organized in a rule tree and performs conditional routing during online inference to retrieve rules whose preconditions are satisfied, thereby generating traceable root cause analyses and repair recommendations under explicit evidence constraints.

Experimental results show that Step 1 consistently improves extraction quality while preserving practical output length, reaching 0.423 F1 and improving recall and F1 over LogSage by about 15.5% and 3.7%, respectively. The full Unravel pipeline achieves the strongest overall RCA performance, reaching 84.0% with DeepSeekV3 and outperforming MCTS by 8.7% while reducing runtime by about 61.0%. These findings demonstrate that combining evidence-preserving extraction with rule-constrained reasoning yields clear gains in both diagnostic quality and efficiency. Future work will focus on improving incremental update and deduplication mechanisms for the rule knowledge base, extending adaptation to additional architectures and build systems, and integrating the framework into developer-facing tools (e.g., IDE plugins) to support practical deployment in production engineering workflows. Our dataset and tools will be made publicly available upon acceptance to support reproducibility.

6 Threats to Validity

Internal validity. The RCA scoring rubric relies on expert judgment. To mitigate subjectivity, each case is independently assessed by three domain experts and the final score is determined by majority vote; the resulting Fleiss' κ of 0.72 indicates substantial agreement. The rule-tree construction also involves LLM-based structuring, whose outputs undergo validation by an automated schema checker and a rewrite loop, thereby reducing extraction errors.

External validity. Our dataset is drawn from a single OBS instance focused on RISC-V. Although the 127 cases span five error categories and four build stages, they may not fully represent all failure patterns encountered in other distributions or architectures. We plan to extend the evaluation to additional ecosystems (e.g., Debian, Fedora) and architectures (e.g., LoongArch) in future work.

Construct validity. The composite score equally weights four sub-dimensions. Different weighting schemes could shift relative rankings. We adopt equal weights for simplicity and report per-dimension scores in Table 3 and Figure 6 to allow readers to assess each dimension independently.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Song Chen and Hai Liao. 2022. Bert-log: Anomaly detection for system logs based on pre-trained language model. *Applied Artificial Intelligence* 36, 1 (2022), 2145642.

[3] Enfang Cui, Tianzheng Li, and Qian Wei. 2023. Risc-v instruction set architecture extensions: A survey. *IEEE Access* 11 (2023), 24696–24711.

[4] Hetong Dai, Heng Li, Che-Shao Chen, Weiyi Shang, and Tse-Hsun Chen. 2020. Logram: Efficient log parsing using *n* n-gram dictionaries. *IEEE transactions on software engineering* 48, 3 (2020), 879–892.

[5] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 1285–1298.

[6] Wei Guan, Jian Cao, Shiyou Qian, Jianqi Gao, and Chun Ouyang. 2024. Logllm: Log-based anomaly detection using large language models. *arXiv preprint arXiv:2411.08561* (2024).

[7] Haixuan Guo, Shuhan Yuan, and Xintao Wu. 2021. Logbert: Log anomaly detection via bert. In *2021 international joint conference on neural networks (IJCNN)*. IEEE, 1–8.

[8] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. 2017. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE international conference on web services (ICWS)*. IEEE, 33–40.

[9] Van-Hoang Le and Hongyu Zhang. 2022. Log-based anomaly detection with deep learning: How far are we?. In *Proceedings of the 44th International Conference on Software Engineering*. 1356–1367.

[10] Jonathan Lee, Mason Li, and Kuo-Hsun Hsu. 2024. Applying transformer models for automatic build errors classification of java-based open source projects. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 282–283.

[11] Xiaoyun Li, Hongyu Zhang, Van-Hoang Le, and Pengfei Chen. 2024. Logshrink: Effective log compression by leveraging commonality and variability of log data. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.

[12] Zeyan Li, Chengyang Luo, Yiwei Zhao, Yongqian Sun, Kaixin Sui, Xiping Wang, Dapeng Liu, Xing Jin, Qi Wang, and Dan Pei. 2019. Generic and robust localization of multi-dimensional root causes. In *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 47–57.

[13] Yilun Liu, Shimin Tao, Weibin Meng, Feiyu Yao, Xiaofeng Zhao, and Hao Yang. 2024. Logprompt: Prompt engineering towards zero-shot and interpretable log analysis. In *Proceedings of the 2024 IEEE/ACM 46th international conference on software engineering: Companion proceedings*. 364–365.

[14] Yiling Lou, Zhenpeng Chen, Yanbin Cao, Dan Hao, and Lu Zhang. 2020. Understanding build issue resolution in practice: symptoms and fix patterns. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 617–628.

[15] Zeyang Ma, An Ran Chen, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. 2024. Llmparser: An exploratory study on using large language models for log parsing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[16] Paolo Notaro, Soroush Haeri, Jorge Cardoso, and Michael Gerndt. 2023. Logrule: Efficient structured log mining for root cause analysis. *IEEE Transactions on Network and Service Management* 20, 4 (2023), 4231–4243.

[17] Jiaxing Qi, Shaohan Huang, Zhongzhi Luan, Shu Yang, Carol Fung, Hailong Yang, Depei Qian, Jing Shang, Zhiwen Xiao, and Zhihui Wu. 2023. Loggpt: Exploring chatgpt for log-based anomaly detection. In *2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. IEEE, 273–280.

[18] Thomas Rausch, Waldemar Hummer, Philipp Leitner, and Stefan Schulte. 2017. An empirical analysis of build failures in the continuous integration workflows of java-based open-source software. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 345–355.

[19] Devjeet Roy, Xuchao Zhang, Rashi Bhave, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. Exploring llm-based agents for root cause analysis. In *Companion proceedings of the 32nd ACM international conference on the foundations of software engineering*. 208–219.

[20] Weipeng Shuai, Jie Liu, Zhirou Ma, Liangyi Kang, Zehua Wang, Shuai Wang, Dan Ye, Hui Li, Wei Wang, and Jiaxin Zhu. 2025. Root Cause Analysis of RISC-V Build Failures via LLM and MCTS Reasoning. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2772–2782.

[21] Yicheng Sui, Yuzhe Zhang, Jianjun Sun, Ting Xu, Shenglin Zhang, Zhengdan Li, Yongqian Sun, Fangrui Guo, Junyu Shen, Yuzhi Zhang, et al. 2023. Logkg: Log failure diagnosis through knowledge graph. *IEEE Transactions on Services Computing* 16, 5 (2023), 3493–3507.

[22] Risto Vaarandi. 2008. Mining event logs with slct and loghound. In *NOMS 2008-2008 IEEE Network Operations and Management Symposium*. IEEE, 1071–1074.

[23] Carmine Vassallo, Gerald Schermann, Fiorella Zampetti, Daniele Romano, Philipp Leitner, Andy Zaidman, Massimiliano Di Penta, and Sebastiano Panichella. 2017. A tale of CI build failures: An open source and a financial organization perspective. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 183–193. doi:10.1109/ICSME.2017.67

[24] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2024. Rcagent: Cloud root cause analysis by autonomous agents with tool-augmented large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4966–4974.

[25] Thorsten Wittkopp, Philipp Wiesner, and Odej Kao. 2024. Logrca: Log-based root cause analysis for distributed services. In *European Conference on Parallel Processing*. Springer, 362–376.

[26] Junjielong Xu, Ruichun Yang, Yintong Huo, Chengyu Zhang, and Pinjia He. 2024. Divlog: Log parsing with prompt enhanced in-context learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.

[27] Weiyuan Xu, Juntao Luo, Tao Huang, Kaixin Sui, Jie Geng, Qijun Ma, Isami Akasaka, Xiaoxue Shi, Jing Tang, and Peng Cai. 2025. LogSage: An LLM-based framework for CI/CD failure detection and remediation with industrial validation. *arXiv preprint arXiv:2506.03691* (2025).

[28] Lin Yang, Junjie Chen, Zan Wang, Weijing Wang, Jiajun Jiang, Xuyuan Dong, and Wenbin Zhang. 2021. Plelog: Semi-supervised log-based anomaly detection via probabilistic label estimation. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 230–231.

[29] Ding Yuan, Haohui Mai, Weiwei Xiong, Lin Tan, Yuanyuan Zhou, and Shankar Pasupathy. 2010. Sherlog: error diagnosis by connecting clues from run-time logs. In *Proceedings of the fifteenth International Conference on Architectural support for programming languages and operating systems*. 143–154.

[30] Dylan Zhang, Xuchao Zhang, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2023. PACE-LM: Prompting and Augmentation for Calibrated Confidence Estimation with GPT-4 in Cloud Incident Root Cause Analysis. *arXiv preprint arXiv:2309.05833* (2023).